

Bellman-Ford: Correct Even With Negative Weights

Programming and Data Structures — Weeks 12-13, Lecture 3

Dr. Innocent Nyalala

Objectives

By the end of this lecture, you will be able to implement Bellman-Ford's algorithm from scratch, and explain why it needs no priority queue at all, unlike Lecture 1's Dijkstra's; trace it completely on Week 11 Lecture 4's exact negative-weight counterexample, confirming it produces the correct answer where Dijkstra's algorithm failed; detect a negative-weight cycle and explain precisely why shortest paths become undefined, not merely difficult, once one exists; and state Bellman-Ford's exact cost, explaining the genuine trade-off it makes against Dijkstra's algorithm.

Outline

Today covers a rumor-spreading analogy motivating repeated relaxation; Bellman-Ford's core idea of relaxing every edge repeatedly, a bounded number of times; a complete trace on Week 11 Lecture 4's exact negative-weight counterexample; a proof that $|V| - 1$ rounds always suffice; detecting a negative-weight cycle using one additional round; Bellman-Ford's exact $O(VE)$ cost and its genuine trade-off against Dijkstra's algorithm; the MTech-track explanation of why Bellman-Ford needs no priority queue at all, structurally; an in-class quiz; and a summary.

The rumor-spreading analogy

Picture a rumor spreading through a social network, where a shorter, more refined version of the story can genuinely arrive later and correct an earlier, longer version someone already believed. Rather than trusting and permanently finalizing the first version heard, as Dijkstra's greedy finalization does, everyone in this analogy simply keeps updating their version of the story whenever a shorter one arrives, from any source. After enough rounds of this repeated gossip, every person's version has genuinely stabilized — this is Bellman-Ford's entire strategy, made concrete before the formal algorithm.

Bellman-Ford's core idea: relax every edge, repeatedly

```
def bellman_ford(edges, start, n):
    # edges: list of (u, v, weight) tuples
    dist = {start: 0}
    prev = {start: None}
    for _ in range(n - 1):
        # exactly n-1 rounds - proven sufficient bel
        for u, v, weight in edges:
            if u in dist and dist[u] + weight < dist.get(v, float('inf')):
                dist[v] = dist[u] + weight
                prev[v] = u
                # "relax" the edge - found a better path
    return dist, prev
```

“Relaxing” an edge means checking whether traveling through it would improve the currently known distance to its destination — exactly Dijkstra’s own update rule from Lecture 1, reused here unchanged. The genuine structural difference is the complete absence of a priority queue or a visited set: every edge is simply checked, in a fixed order, for a fixed number of rounds, with no greedy finalization occurring at any point. A node’s distance can improve on any round, including the very last one, unlike Dijkstra’s permanent, one-time finalization — this is precisely the property that allows Bellman-Ford to remain correct even when edge weights are negative.

Tracing Bellman-Ford on Week 11’s negative-weight counterexample

Edges: (0,1,1), (0,2,4), (2,1,-10) -- Week 11 Lecture 4's exact graph, n=3 nodes, 2 rounds

Round 1:

edge(0,1,1): $\text{dist}[0]=0$, $0+1=1 < \text{inf} \rightarrow \text{dist}[1]=1$
edge(0,2,4): $\text{dist}[0]=0$, $0+4=4 < \text{inf} \rightarrow \text{dist}[2]=4$
edge(2,1,-10): $\text{dist}[2]=4$, $4+(-10)=-6 < \text{dist}[1]=1 \rightarrow \text{dist}[1]=-6$ <-- IMPROVED, unlike Dijkstra

Round 2 (n-1=2 rounds total):

edge(0,1,1): $0+1=1$, NOT $< -6 \rightarrow$ no change
edge(0,2,4): $0+4=4$, NOT $< 4 \rightarrow$ no change
edge(2,1,-10): $4+(-10)=-6$, NOT $< -6 \rightarrow$ no change (already optimal)

Final: $\text{dist}=\{0:0, 1:-6, 2:4\}$

Tracing Bellman-Ford on the exact graph that broke Dijkstra’s algorithm in Week 11 Lecture 4: in round 1, edges are relaxed in order, initially setting $\text{dist}[1] = 1$ via the direct edge, then discovering, via the edge from node 2, that $\text{dist}[1]$ can be improved to -6 — critically, this improvement is accepted,

since Bellman-Ford never finalizes a node permanently the way Dijkstra's does. Round 2 confirms no further improvement is possible, and the algorithm correctly reports `dist[1] = -6`, the true shortest distance, directly resolving the exact failure Week 11 Lecture 4 demonstrated for Dijkstra's algorithm on this identical graph.

Why $|V| - 1$ rounds suffice — proven, not assumed

The claim that $|V| - 1$ rounds always suffice rests on a precise argument. Any genuine shortest path between two nodes, in a graph with no negative-weight cycle, uses at most $|V| - 1$ edges — a path revisiting any node would contain a cycle, and removing that cycle (assuming it does not have negative total weight) can only shorten the path, contradicting it being shortest in the first place. Each complete round of relaxing every edge is guaranteed to correctly extend the known shortest-path estimate by at least one additional edge, for every node whose true shortest path has not yet been fully accounted for. After $|V| - 1$ rounds, every shortest path, each using at most $|V| - 1$ edges, has therefore been fully and correctly computed — no further round can improve any distance, which is exactly what the next slide's cycle-detection method checks directly.

Reconstructing the path, exactly Lecture 1's technique

```
def reconstruct_path(prev, target):
    path = []
    node = target
    while node is not None:
        path.append(node)
        node = prev[node]
    return path[::-1]

path_to_1 = reconstruct_path(prev, 1)
print(path_to_1)    # [0, 2, 1] - same reconstruction technique as Dijkstra's
```

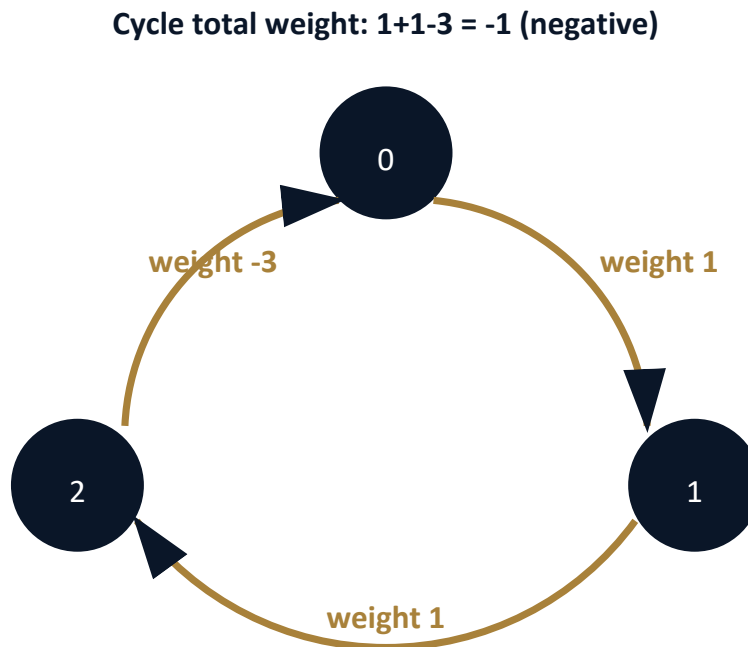
Path reconstruction uses exactly the same technique as Lecture 1's Dijkstra's algorithm: walk the `prev` array backward from the target to the start, then reverse. On this lecture's traced example, this correctly recovers `[0, 2, 1]` — the path achieving distance `-6`, confirming Bellman-Ford's `prev` tracking works identically to Dijkstra's despite the underlying algorithm being structurally quite different.

Detecting a negative-weight cycle: one more round

```
def has_negative_cycle(edges, dist):
    for u, v, weight in edges:
        if u in dist and dist[u] + weight < dist.get(v, float('inf')):
            return True      # improvement STILL possible after n-1 rounds - impossible without
    return False
```

After exactly $|V| - 1$ rounds of relaxation, no genuine shortest path should be improvable any further, since every real shortest path uses at most $|V| - 1$ edges, per the previous slide's proof. Running one additional round of relaxation and finding a further improvement is therefore only possible if a negative-weight cycle exists somewhere in the graph — the reported “shortest path” can keep getting cheaper indefinitely by looping through that cycle again and again, each loop subtracting more total weight. This is precisely why a negative-weight cycle makes the shortest-path problem genuinely undefined for affected nodes, not merely computationally difficult: no true minimum distance exists at all, since it can always be decreased further by one more trip around the cycle.

The picture: a negative cycle, and why distance keeps shrinking



Each full loop around this cycle subtracts more distance — no minimum exists for nodes reachable from the cycle

Each full traversal of this cycle subtracts more total weight from the accumulated distance — there is no lower bound, and no number of Bellman-Ford rounds, however large, would ever converge to a

stable answer for nodes reachable through this cycle.

Cost: $O(VE)$, and the genuine trade-off against Dijkstra's

This table states the genuine trade-off plainly: Dijkstra's algorithm is asymptotically faster, at $O((V + E) \log V)$, but is fundamentally unsound on graphs with negative weights, with no mechanism to even detect the problem. Bellman-Ford costs $O(VE)$, slower for the sparse, non-negative-weight graphs where Dijkstra's applies, but correctly handles negative weights and directly detects negative-weight cycles as a byproduct. Neither algorithm dominates the other — the correct choice depends entirely on whether negative weights are possible in the specific problem at hand, exactly the pattern-recognition discipline this course has built since Week 8 Lecture 4.

MTech addition — why no priority queue is needed, structurally

Dijkstra's algorithm needs a priority queue specifically because its correctness proof, established in Lecture 1, depends critically on always processing the currently closest unvisited node next, in strict order. Bellman-Ford makes no such ordering claim whatsoever — it simply relaxes every edge, in any fixed order, for a fixed number of rounds, regardless of which nodes currently appear closest. This is the precise structural reason negative weights break Dijkstra's algorithm, whose correctness proof depends on an ordering assumption that negative weights violate, while leaving Bellman-Ford entirely unaffected, since it never makes that ordering assumption in the first place.

Tracing negative-cycle detection completely

Cycle: edges $(0,1,1)$, $(1,2,1)$, $(2,0,-3)$ -- total cycle weight: $1+1+(-3) = -1$, genuinely negative

After $n-1=2$ rounds ($n=3$ nodes): dist stabilizes to some values

Round 3 (the EXTRA check round): relax $(2,0,-3)$: if $\text{dist}[2]+(-3) < \text{dist}[0]$, IMPROVEMENT FOUND
→ `has_negative_cycle` returns True

Tracing negative-cycle detection on a genuine 3-node cycle whose total weight is $1 + 1 + (-3) = -1$, negative: after the standard $n - 1 = 2$ rounds, one additional relaxation pass finds that the edge from node 2 back to node 0 still offers an improvement, since traversing the full cycle continues to reduce the accumulated distance. This extra-round improvement is exactly the signal `has_negative_cycle` checks for, correctly identifying that no true shortest-path distance exists for nodes reachable through this cycle.

Exercise

As an exercise, implement `bellman_ford` and `has_negative_cycle` exactly as presented in this lecture, verifying against the traced example: `dist = {0: 0, 1: -6, 2: 4}`. Separately, construct a small graph containing a genuine negative-weight cycle of your own design, and confirm that `has_negative_cycle` correctly detects it after running the main algorithm's $|V| - 1$ rounds.

Real-world use of Bellman-Ford

Bellman-Ford is genuinely deployed, not merely a theoretical counterpoint to Dijkstra's algorithm. Currency arbitrage detection, previewed in Week 11 Lecture 4, models currency exchange rates as edge weights using $-\log(\text{rate})$, transforming multiplicative currency conversion into additive edge weights; a negative-weight cycle detected by Bellman-Ford directly corresponds to a genuine arbitrage opportunity — converting through a sequence of currencies and ending up with strictly more money than started. RIP, an older internet routing protocol still used in some contexts, uses a Bellman-Ford-style distance-vector approach. Any setting where costs can genuinely be negative — refunds, penalties, or gains — and Dijkstra's non-negative assumption cannot be guaranteed, is a candidate for this lecture's algorithm instead.

Quiz — how many rounds are needed?

Given a graph with 6 nodes, determine, using this lecture's proof, the minimum number of edge-relaxation rounds Bellman-Ford needs to guarantee correct shortest distances, assuming no negative-weight cycle is present.

A second worked trace — a graph with no negative cycle, confirmed stable

Edges: (0,1,4), (0,2,1), (2,1,2), (1,3,1), (2,3,5) -- Lecture 1's second worked example, n=4

3 rounds (n-1=3):

Round 1: `dist[1]=4` (via 0,1), `dist[2]=1` (via 0,2), then `dist[1]` improves to `1+2=3`, `dist[3]=1` (via 1,3)

Round 2: `dist[3]` improves to `3+1=4` (via node 1)

Round 3: no further changes

Extra round (cycle check): relax every edge again - NO improvement found → no negative cycle

Final: `dist={0:0, 1:3, 2:1, 3:4}` - EXACTLY matches Lecture 1's Dijkstra's result

Running Bellman-Ford on the identical 4-node example traced with Dijkstra's algorithm in Lecture 1 produces exactly the same final distances, $\{0:0, 1:3, 2:1, 3:4\}$ — confirming both algorithms agree on this positive-weight graph, exactly as expected, since Dijkstra's correctness proof applies here. The extra cycle-check round correctly finds no further improvement, confirming this graph contains no negative-weight cycle, as its all-positive edge weights already guarantee.

Quiz — answer

The minimum number of rounds is $|V| - 1 = 6 - 1 = 5$. This follows directly from this lecture's proof: any genuine shortest path in a 6-node graph, with no negative-weight cycle, uses at most 5 edges, since a path with 6 or more edges would necessarily revisit a node, containing a removable cycle. Five complete rounds of relaxation are therefore guaranteed sufficient to correctly compute every shortest distance in this graph.

A closing note on the decision between the two algorithms

Applying Week 10 Lecture 4's decision framework directly to this specific choice: the single deciding question is whether edge weights can ever genuinely be negative in the problem at hand. If genuinely never — physical road distances or travel times, for instance — Dijkstra's algorithm is the correct choice, for its strictly better asymptotic cost. If negative weights are possible, or the input's non-negativity cannot be independently verified — financial modeling, penalty or refund systems, or any data source not fully under the programmer's control — Bellman-Ford is the correct choice, prioritizing correctness over speed. This decision should be made explicitly and verified, never assumed by convenience or by which algorithm happens to be more familiar.

A closing note on early termination

A practical optimization worth noting: if no edge relaxation succeeds during an entire round, the algorithm has already fully converged, and every subsequent round would perform no useful work at all. Tracking whether any relaxation occurred during a round, and stopping early if none did, is a simple and valid optimization, though it does not change the algorithm's worst-case asymptotic cost. This directly mirrors Week 5 Lecture 1's early-stop optimization for bubble sort, which similarly detected when a full pass produced no swaps and terminated early — the identical optimization idea, recurring here in a structurally different algorithm.

Summary

Bellman-Ford relaxes every edge repeatedly, for exactly $|V| - 1$ rounds, using no priority queue and no permanent node finalization, which is precisely what allows it to remain correct with negative

edge weights. This directly resolves Week 11 Lecture 4's negative-weight counterexample, verified by an exact trace matching the true shortest distance. Finding a further improvement possible after $|V| - 1$ rounds proves a negative-weight cycle exists, at which point shortest path becomes genuinely undefined for affected nodes, not merely computationally difficult. Bellman-Ford's $O(VE)$ cost trades speed for correctness on negative weights — neither Bellman-Ford nor Dijkstra's algorithm dominates the other unconditionally, and the correct choice depends on whether negative weights are possible in a given problem. This closes this unit's sequence of shortest-path algorithms; next lecture returns to minimum spanning trees, completing Week 10 Lecture 3's brief preview with Prim's algorithm covered in full.